

Table of Content :

Question A:	2
Question B:	3
Question C:	4
Question D:	6
Question E:	7
Questions.	8
Task No.7	9
Remaining tasks:	10

Table of Figure :

Figure 1: Basic Hello World Program	3
Figure 2: Program with user Input.	4
Figure 3: Creating and editing a C program in ubuntu	5
Figure 4: GCC installation Commands	6
Figure 5: GCC commands for testing stages of compilation.	7
Figure 6:Preprocessing stage	8
Figure 7: C file creation for a value input	9
Figure 8: commands for creation and compilation of a C file.	9
Figure 9: Compilation of a C file and its 0 output verification	11
Figure 10: Verifying the user input C file.	11

Question A:



```
root@Ubuntu: ~ — sudo -i
GNU nano 8.7.1 task.c
#include <stdio.h>

int main(){
    printf("Hello, World!\n");
    return 0;
}
```

Trash

Figure 1: Basic Hello World Program

Summary: A C file was created and a basic program printing out “Hello World” was written.

Question B:



The image shows a terminal window with a dark background. At the top, the date and time 'Sep 18 15:35' are displayed. Below that, the prompt 'root@Ubuntu: ~ — sudo -i' is visible. The terminal is running the GNU nano 8.7.1 text editor, editing a file named 'task1.c'. The code in the editor is a C program that prompts the user for a number and prints it back. The code is as follows:

```
GNU nano 8.7.1 task1.c
#include <stdio.h>

int main(){
    int number;

    printf("Enter a number : ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 0;
}
```

Figure 2: Program with user Input.

Summary: A program which demonstrates how the printf & scanf for input and output operations was written.

Question C:



```
GNU nano 8.7.1 task2.c
#include <stdio.h>

int main(){
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n-- User Info ---\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("height: %.2f m\n", height);

    return 0;
}
```

Figure 3: Creating and editing a C program in ubuntu

Summary: A C file was created and use of different format specifiers and variables was tested.

```
root@Ubuntu:~/task_18sept# apt install build-essential
build-essential is already the newest version (12.12ubuntu2.26.04.2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 11
root@Ubuntu:~/task_18sept# gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

root@Ubuntu:~/task_18sept#
```

Figure 4: GCC installation Commands

Summary: Different GCC commands were performed for its installation and its version check.

Question D:

```
root@Ubuntu:~/task_18sept# mv task.c hello.c
root@Ubuntu:~/task_18sept# gcc -E hello.c -o hello.i
root@Ubuntu:~/task_18sept# gcc -s hello.i -o hello.s
root@Ubuntu:~/task_18sept# gcc -c hello.s -o hello.o
hello.s: Assembler messages:
hello.s: Warning: end of file not at end of a line; newline inserted
hello.s:1: Error: junk at end of line, first unrecognized character valued 0x7f
hello.s:1: Error: junk at end of line, first unrecognized character valued 0x3
hello.s:1: Error: junk at end of line, first unrecognized character is '>'
hello.s:1: Error: junk at end of line, first unrecognized character valued 0x1
hello.s:1: Error: junk at end of line, first unrecognized character is ''
hello.s:1: Error: junk at end of line, first unrecognized character is '@'
hello.s:1: Error: junk at end of line, first unrecognized character is '@'
hello.s:1: Error: junk at end of line, first unrecognized character is '@'
hello.s:1: Error: junk at end of line, first unrecognized character is '8'
hello.s:1: Error: junk at end of line, first unrecognized character valued 0xe
```

Figure 5: GCC commands for testing stages of compilation.

Summary: The commands which were learned in our lecture were tested in a c file.

Question E:

```
Sep 18 15:54
root@Ubuntu: ~/task_18sept — sudo -i

# 0 "hello.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "hello.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 431 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 432 "/usr/include/features.h" 2 3 4
# 539 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 540 "/usr/include/features.h" 2 3 4
# 563 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4
# 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4
# 11 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 2 3 4
# 564 "/usr/include/features.h" 2 3 4
# 34 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 2 3 4
# 29 "/usr/include/stdio.h" 2 3 4
:
```

Figure 6:Preprocessing stage

Questions.

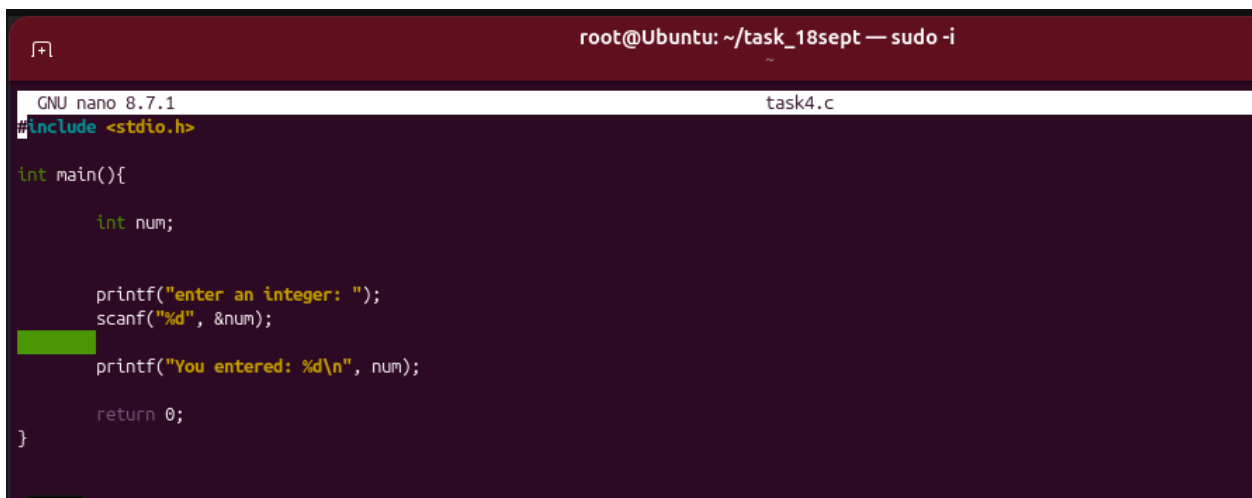
Q: Explain the difference between the four stages of compilation. What file extension is produced at each stage?

→ Preprocessing (.i) → Compilation (.s) → Assembly (.o) → Linking executable .exe on Windows / a.out or custom name on Linux

Q: What does a return value of 0 from the main() function indicate to the operating system? What about a non-zero return value?

→ 0 indicates the program executed successfully, while a non-zero value indicates an error or abnormal termination.

Q: Writing a C program with the given conditions.



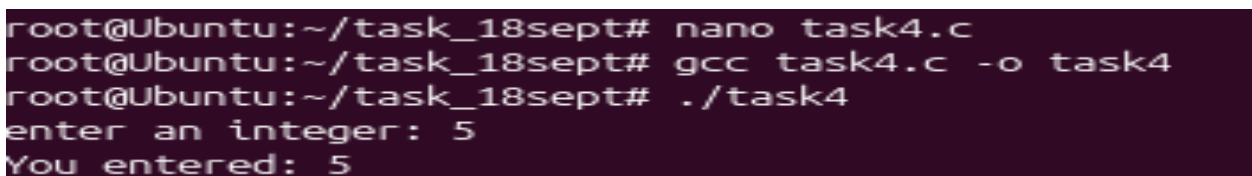
```
root@Ubuntu: ~/task_18sept — sudo -i
GNU nano 8.7.1 task4.c
#include <stdio.h>

int main(){
    int num;

    printf("enter an integer: ");
    scanf("%d", &num);
    printf("You entered: %d\n", num);

    return 0;
}
```

Figure 7: C file creation for a value input



```
root@Ubuntu:~/task_18sept# nano task4.c
root@Ubuntu:~/task_18sept# gcc task4.c -o task4
root@Ubuntu:~/task_18sept# ./task4
enter an integer: 5
You entered: 5
```

Figure 8: commands for creation and compilation of a C file.

Q: What GCC command to use?

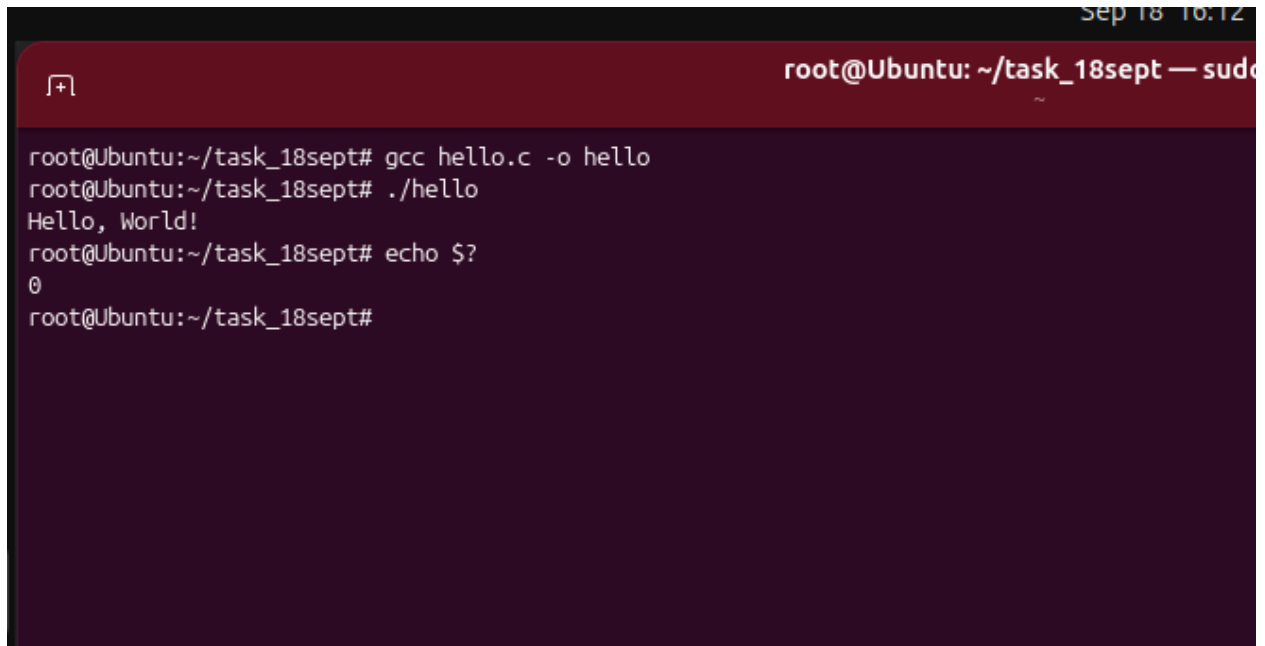
- Only preprocess: `gcc -E program.c`
- Generate assembly: `gcc -S program.c`
- Create object file: `gcc -c program.c`

Q: Explain why C is called a "compiled language" and how this differs from interpreted languages.

- C is called a compiled language because its source code is converted into machine code by a compiler before execution, unlike interpreted languages that execute code through an interpreter at runtime.

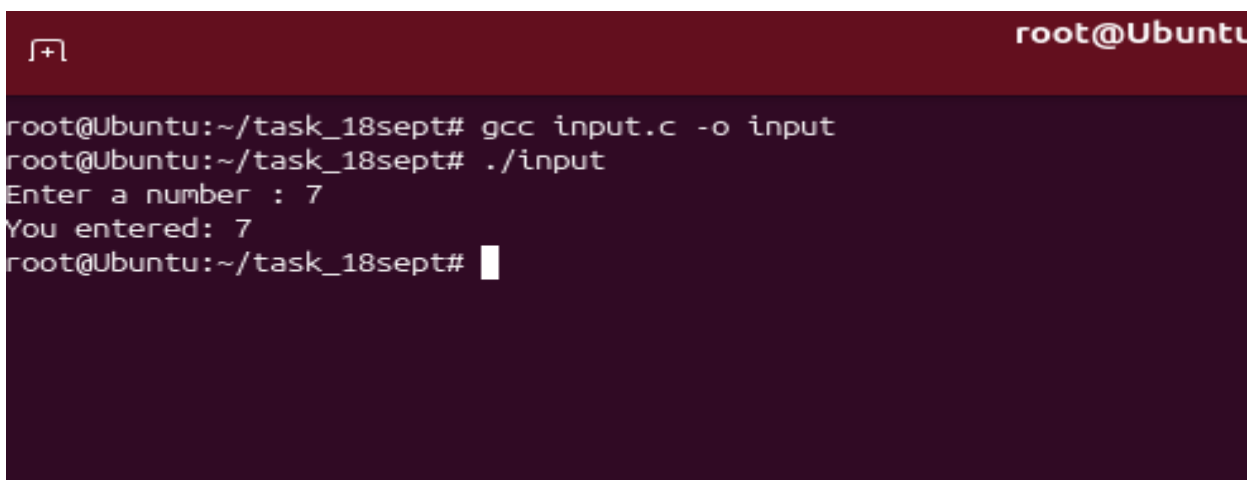
Task No.7

Remaining tasks:

A terminal window with a dark background and a red title bar. The title bar contains a window icon on the left and the text 'root@Ubuntu: ~/task_18sept — sudo' on the right. The terminal shows the following commands and output:

```
root@Ubuntu:~/task_18sept# gcc hello.c -o hello
root@Ubuntu:~/task_18sept# ./hello
Hello, World!
root@Ubuntu:~/task_18sept# echo $?
0
root@Ubuntu:~/task_18sept#
```

Figure 9: Compilation of a C file and its 0 output verification

A terminal window with a dark background and a red title bar. The title bar contains a window icon on the left and the text 'root@Ubuntu' on the right. The terminal shows the following commands and output:

```
root@Ubuntu:~/task_18sept# gcc input.c -o input
root@Ubuntu:~/task_18sept# ./input
Enter a number : 7
You entered: 7
root@Ubuntu:~/task_18sept#
```

Figure 10: Verifying the user input C file.